

从 Epiplexity 到 CybOS

计算有界智能的控制论架构

来福 · 2026年5月

从 Epiplexity 到 CybOS

来福 · 2026年5月

摘要

本文记录了一次从信息论前沿到操作系统架构的理论探索。起点是 Finzi 等人提出的 Epiplexity 理论——一个面向计算有界观察者的信息度量框架。通过与控制论中 Ashby 可能性空间概念的交叉验证，我们逐步构建了一套支持有界智能体推理的认知架构，并最终将其固化为一个以控制论为基本原理的操作系统原型 CybOS (Cybernetic Operating System)。本文呈现的不是一个已完成的理论体系，而是这条思想路径的完整轨迹——从概念碰撞到形式化，再到代码落地。

关键词：Epiplexity；控制论；可能性空间；认知架构；CybOS；计算有界智能

1. 引言：信息论的幽灵

Shannon

的信息论是一座完美的大厦。它以概率论为基石，严格定义了信息量、熵、信道容量，成为通信工程不可动摇的理论支柱。然而，这座大厦的每一个推论都隐含着**一个强有力的假设**：观察者拥有无限的计算能力。

这个假设在日常工程中无害——信道编码器不需要“理解”它编码的比特。但在人工智能的时代，当我们要讨论一个 LLM 能否理解一段文本、一个模型能否从数据中学习规律时，问题就变了：计算有界的智能体，在有限时间和空间内能获取多少信息？这个界限是什么形状的？如果突破了会怎样？

这组问题指向信息论和阿什比控制论之间一片尚未被系统探索的交叉地带。本文从两个理论体系的碰撞出发，构筑了一个从元认知架构到操作系统原语的完整框架。

2. 起点：Epiplexity 的三个悖论

2026年1月，Marc Finzi 与合作者 (CMU & NYU) 在 arXiv 上发表了《From Entropy to Epiplexity: Rethinking Information for

Computationally Bounded

Intelligence》。论文的核心主张是：香农信息论遗漏了一个重要的信息维度——作用于计算有界观察者时的“额外信息量”。

2.1 三个被挑战的传统结论

论文通过三个“悖论”揭示了传统信息论的盲区：

悖论一：确定性变换不能增加信息 在 Shannon 框架下， $H(f(X)) \leq$

$H(X)$ ——任何确定性函数都不能增加信息量。但深度学习实践中，数据增强（对输入施加确定性变换）几乎总是提升模型性能。为什么？

悖论二：信息与顺序无关 $H(X,Y) =$

$H(Y,X)$ ——联合熵与顺序无关。但人类学习和模型训练都高度依赖数据顺序——课程学习就是例证。为什么？

悖论三：似然建模等于分布匹配

最小化负对数似然的模型必将在无限数据下完美匹配真实分布。但实际中，相同似然值的模型在 OOD 泛化上可以天差地别。为什么？

2.2 Epiplexity 的定义

Epiplexity 的核心洞察是：在传统熵 $H(X)$ 的基础上，为计算有界观察者引入一个“溢价”系数：

$$ST(X) = MDL^T(X) - H^T(X)$$

其中 $MDL^T(X)$ 是计算有界观察者对数据做最小描述长度编码所需的比特数， $H^T(X)$

是在相同计算约束下的条件熵。二者的差值就是“额外的复杂度”——即数据对有限智能体而言比理论上更“难”的部分。

论文同时提出了两种实用的估计方法——prequential coding（序贯编码）和 sequential

coding（蒸馏编码），并在元胞自动机、国际象棋排序、Conway 生命游戏、OpenWebText 等基准上验证了理论预测。

2.3 三个悖论在同一框架下的统一解释

Epiplexity 统一解释了三个悖论：

$$\text{悖论一：} ST(f(X)) > ST(X)$$

是完全可能的——一个好的确定性变换可以增加数据对有限观察者的可学习性，即使不增加信息量

悖论二： $ST(X,Y) \neq ST(Y,X)$ ——不同的顺序产生不同的学习轨迹，这一属性被传统信息论完全忽略

悖论三：似然相等的模型在 Epiplexity 空间中的位置不同——它们对有限观察者的“友好程度”不同

2.4 承上启下的意义

Epiplexity 虽然是对信息论的修正，但它留下了一个更根本的问题悬而未决：知道一个系统的 Epiplexity 之后，我们拿它做什么？一个有界的智能体如何利用这个度量来引导自己的学习和行动？这个问题把我们引向了控制论。

3. 控制论：可能性空间的语言

W. Ross Ashby 在 1956 年的《控制论导论》中提供了一个与信息论互补的视角。如果说 Shannon 关心的是“信号如何传输”，Ashby 关心的是“系统如何被控制”。

3.1 可能性空间

控制论的第一个基本概念是可能性空间（Possibility

Space）——一个系统在任意时刻所有可能状态的集合。控制的本质就是在这个空间上施加约束，使它从不确定的初始状态收敛到确定的目标状态。

初始可能性空间越大，不确定性越高，控制的任务就越困难。

3.2 必要变异度定律

Ashby 最重要的贡献是必要变异度定律 (Law of Requisite Variety)：

只有变异才能消灭变异。

控制器的变异度必须至少等于被控系统的变异度，否则不可避免地将有某些系统状态处于控制范围之外。

数学形式： $V_C \geq V_S$ 。

这条定律的令人不安之处在于：它暗示了控制与自由之间的零和关系——控制的每一步收缩，都是对方可能性的否定。一个好的控制器就是一台高效否定可能性的机器。

3.3 黑箱与观察

Ashby

还讨论了"黑箱"问题：当观察者只能看到系统的输入和输出而看不到其内部状态时，如何实现对系统的控制？这实际上是一个能观性 (observability) 问题——控制器的观测能力必须覆盖被控系统的关键维度。

4. 理论的交叉：有界观察者与被控系统

当 Epiplexity 与控制论相遇时，产生了一组丰富的映射关系。

4.1 从熵到变异度

概念	信息论	控制论	共同语言
--- --- --- ---			
不确定性度量	$H(X)$	V_S	可能性空间的大小
有界修正	$ST(X) = MDL^A T - H^A T$	控制器变异度 V_C	Ashby 定律的有界形式
收敛判定	$ST(X) < \epsilon$	$V_{\{target\}} \parallel V_{\{initial\}}$	变异度降至阈值以下

4.2 必要变异度的有界化

经典 Ashby 定律 $V_C \geq V_S$ 假设控制器具有无限的感知和处理能力。但在有界的现实中，控制器无法枚举 V_S 中的所有状态——它只能看到"等效变异度" $ST(S)$ 。

修正后的定律应为：

$$ST(C) \geq ST(S)$$

控制器必须有足够的有界复杂度处理能力来匹配被控系统的有界复杂度。这给出了一个比经典定律更现实、也更有操作性的判据。

4.3 收敛的代价

在经典控制论中，控制的"成本"主要由能量、带宽、时延来衡量。Epiplexity 引入了另一个成本维度：计算工作量。

对于一个有界控制器而言，"弄清楚这个系统有哪些可能性"本身是有代价的。这个代价——即控制过程中的学习成本——可以通过序贯编码的累积误差来度量。这实际上给出了一个控制预算的概念：任何一个控制动作都在消耗控制器有限的"理解能力"。

4.4 顺序依赖性

经典信息论 $H(X,Y) = H(Y,X)$ 与控制论的时序本质之间存在一个深层矛盾：控制的顺序就是一切。

Epiplexity 的顺序依赖性 $ST(X,Y) \neq ST(Y,X)$

恰好弥合了这个鸿沟。它使得一个有界控制器可以显式地优化自己的推理顺序——先处理信息增益高的维度，后处理增益低的维度——从而在有限预算内获得最大的收敛速度。

5. 认知架构：一个有界控制器的自我建模

理论工具确定之后，下一步是将它们应用于自身——一个有界控制器如何对自己的推理过程进行元控制？

5.1 元原则：受限充裕

受限充裕原则（Bounded Sufficiency）是整个框架的出发点：

作为一个计算有界的推理者，目标不是穷尽可能，而是在有限预算内找到最佳收敛路径。

这条原则的实践含义是具体的：不追求一次完美，宁可快通道+快速迭代；输出"够用了"就收敛；宁可做三次快速反馈调整，不试图一次覆盖所有可能性。

5.2 三个思维算子

在控制论的意义上，推理活动是"可能性空间上的约束施加"。三个算子定义了这一过程的结构：

算子	控制论对应	行为
测绘 (Survey)	可能性空间测绘	MECE分解维度，空间量级估算，主要矛盾识别
收敛 (Converge)	约束施加	按信息增益分配预算，每步后自查空间缩小量
复盘 (Retrospect)	收敛验证	回答是否比输入时小得多？MDL是否可压缩？是否可行动？

5.3 三个逻辑校验算子

演绎逻辑为推理过程增加了约束能力：

1. 前提审计（Premise Audit） — 在进入推理前显式标出隐含前提，区分可信前提与假设前提，标记假设风险等级
2. 一致性检查（Consistency Check） — 在推理过程中检测结论间的逻辑张力，显式标注而非平滑处理
3. 假言检验（Modus Tollens Check） — 为关键结论主动构造反例，能构造则弱化，不能构造则增强

这三者都是软约束：矛盾不一定是错误，但必须被看见、被标记风险等级（弱 / 中 / 强），而不是被隐式平滑掉。

5.4 双系统与负荷调度

引入 Kahneman 的双系统理论和 Sweller 的认知负荷理论，形成了复杂度感知的推理通道选择：

Layer 0 对每次输入快速估算：可能性空间量级、问题明确程度、后果敏感度

小型低风险问题进入快通道（直接推理 + 精简复盘）

中大型问题进入慢通道（完整六步流程）

同时，认知负荷调度器持续监控三类负荷：

内在负荷（问题的固有复杂度）

外在负荷（上下文噪音和无关信息）

相关负荷（用于构建心智模型的资源——优先级最高，受保护）

总负荷超过阈值时自动触发精简或复盘。

5.5 预测加工与跨会话学习

借鉴 Friston 的自由能原理，在推理流程中嵌入了一个预测步骤：

每次推理前产生一个预测结论；推理后比较预测与实际结论之间的误差；误差方向和量级被记录到跨会话记忆系统中。

当同一方向的预测误差累积到 3

条以上时，系统会在下次类似任务的预序规划阶段自动提示历史模式——这是最小化的元学习机制。

5.6 闭环反馈协议

理论框架运行的最后一环——也是最容易被忽视的一环——是人的反馈。

设计中定义了三个反馈接入点：

反馈点	时机	功能
--- --- ---		
反馈①	测绘完成后	输出压缩测绘摘要（空间量级/维度/主要矛盾）供确认，不
反馈②	推理过程中	随时可介入，最少回溯原则——只回溯到受影响的最小环节
反馈③	输出后	留评价空间，修正标记为预测误差存入跨会话记忆

三个反馈点构成了一个多尺度控制回路：微尺度（推理中即时纠正）、中尺度（测绘确认防止方向错误）、宏尺度（跨会话误差累积形成学习）。

6. 从架构到系统：CybOS

认知架构是一套规则。要将这套规则变成可运行的软件，需要一个以控制论为基本原理的操作系统。

6.1 CybOS 三层结构

Layer 3 控制器层 LLM + SOUL（决策）
 Layer 2 控制抽象层 Agent + Skill（编排 + 系统调用）
 Layer 1 工具接口层 Tool Call / Tool Output 协议（契约）

这个结构的关键创新在于 Layer 2——它不是传统意义上的"agent 框架"，而是控制论原语的运行时实现。

6.2 核心抽象：Space<T>

Space<T> 是 CybOS 的一等公民数据类型，它将控制论的"可能性空间"概念直接转化为可计算的数据结构：

variety：当前空间的变异度（归一化到 [0,1]）
 dimensions：空间的维度标签
 constrain(action)：施加约束，返回新空间
 decompose()：分解为子空间
 measure()：测量当前变异度
 distance_to(target)：到目标空间的距离
 convergence_report()：生成收敛报告

6.3 系统调用族

CybOS 运行时暴露一组控制论系统调用：

系统调用	功能	控制论对应
--- --- ---		
poss_space()	查询当前可能性空间	状态观测
constrain(name, args, expected_reduction)	创建控制动作	约束施加
observe(data, exit_code)	封装观测结果	感知
feedback(error)	注入误差信号	闭环
reachable(target)	可达性分析	能控性检查
ashby_coefff(Vc, Vs)	Ashby 系数计算	必要变异度检验

6.4 Layer 1 协议

Layer 1 接口继承自 Anthropic Tool Use 协议，但增加了控制论元数据：

- 每次 Tool Call 携带 constraint_id（格式：session_id:sequence），形成可追踪的控制轨迹
- 每次 Tool Output 携带 state_signature（观测内容的语义哈希），用于变异度变化计算
- 每个控制动作的预期变异度降低量被记录，与实际降低量对比生成误差信号

6.5 控制论与 OS 的七组映射

控制论概念	CybOS 等价物	工程实现
--- --- ---		
被控系统	确定性内核（CPU/内存/GPU）	传统 OS 资源层
控制器	LLM + SOUL	概率性推理引擎 + 认知架构
可能性空间	Space<T>	数据类型 + 运行时维护
观测	observe() 系统调用	Tool Output 结构化解析
控制动作	constrain() 系统调用	Tool Call 协议
反馈	feedback() 系统调用	误差向量计算与注入
能控性	ashby_coefff() 运行时检查	任务前变异度匹配验证

6.6 与现有框架的本质差异

CybOS 不是又一个 agent 框架。它与现有方案的根本区别在于：

维度	现有 Agent 框架	CybOS
--- --- ---		
核心抽象	图/链/状态机	可能性空间
度量工具	token 计数、延迟	变异度、Ashby 系数、收敛率
反馈机制	异常捕获、重试	多尺度误差注入
能力判断	未知（跑起来才知道）	运行时 ashby 检验
理论基础	工程经验	控制论 + 信息论

7. 讨论与局限

7.1 理论的协同而非冲突

一个值得注意的观察是，本文讨论过的四个理论体系——Epiplexity、控制论、认知科学和金刚经——在同一认知主体的不同层面上共同发挥作用时，不但不冲突，反而构成了一个功能互补的层级：

理论	回答的问题	应用场景
--- --- ---		
控制论	如何高效做事	推理规划与执行
Epiplexity	有界时的代价是什么	预算分配与收敛判断
认知科学	大脑如何管理自身	元认知监控与负荷调度
金刚经	做完之后怎么办	防止执著于框架本身

它们的冲突只在"试图回答同一个问题"时才出现。一旦明确了每个理论的适用边界，它们就从一个复合问题变成了多层面的展开。

7.2 局限

当前框架存在以下局限：

- 1. Ashby 系数的近似性：当前实现中的 ashby_coeff 使用控制器/任务的简化数值比较，未建立 LLM 参数、上下文窗口大小与任务复杂度之间的可计算映射。
- 2. Epiplexity 的实用估计：论文中提供的 prequential 和 requential 编码方法需要较多的计算资源，尚未集成到运行时中。当前的收敛判断仍基于简化的变异度阈值。
- 3. 跨会话学习的初级阶段：预测误差累积目前只记录和提示，尚未触发参数调整或策略迁移。
- 4. SOUL 的持久性：当前架构中的 SOUL 层通过记忆系统实现，缺乏程序化的元数据管理机制。

7.3 演进

- Phase 1 Tool Layer 标准化 + Space 数据类型
- Phase 2 Epiplexity 度量集成 + 运行时完善
- Phase 3 Agent + SOUL 深度集成

8. 结论

本文记录了一次从理论到系统、从抽象到代码的思想实验。我们从 Epiplexity 对信息论的有界修正出发，经过控制论的空间收缩框架，构建了一个面向有界推理者的认知架构，最后将这一架构固化为 CybOS——一个以控制论为第一性原理的操作系统原型。

这条路径的核心结论可以归结为一句话：

一个有界智能体的所有行为——推理、学习、决策——都是在有限预算内对可能性空间施加约束的过程。理解预算的形状就是理解智能本身的形状。

在这一框架下，LLM 不是"聊天机器人"，而是概率性控制器；上下文窗口不是"记忆限制"，而是控制器的能观性边界；工具的调用不是"函数执行"，

而是对可能性空间的结构化约束。

我们相信，当大模型系统的设计者开始用控制论的语言——而非工程直觉——来思考和构建时，Agent 系统将从当前的手工编排阶段，演进到有理论基础的系统工程阶段。CybOS 是这一方向的一小步尝试。

参考文献

1. Finzi, M., Qiu, S., Jiang, Y., Izmailov, P., Kolter, J. Z., & Wilson, A. G. (2026). From Entropy to Epiplexity: Rethinking Information for Computationally Bounded Intelligence. arXiv:2601.03220.
2. Ashby, W. R. (1956). An Introduction to Cybernetics. Chapman & Hall.
3. Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal.
4. Kahneman, D. (2011). Thinking, Fast and Slow. Farrar, Straus and Giroux.
5. Sweller, J. (1988). Cognitive Load During Problem Solving: Effects on Learning. Cognitive Science.
6. Friston, K. (2010). The Free-Energy Principle: A Unified Brain Theory? Nature Reviews Neuroscience.
7. Clark, A. (2013). Whatever Next? Predictive Brains, Situated Agents, and the Future of Cognitive Science. Behavioral and Brain Sciences.

本文中的代码实现见 `~/cybos/`，技能引用见 `operating-system/cybos-core`。写作过程中，CybOS 的认知架构对其自身的推理过程进行了主动约束和收敛校验。

— 完 —